Assignment 4 - Динамическое программирование и обработка строк

Редакция 2.1.0. Создайте свой репозиторий из официального GitHub template; сдайте ссылку точного коммита в Moodle.

Курс: Design and Analysis of Algorithms. **Оценка:** практика 50 + индивидуальная защита-квиз 50 = 100. Вес A4 - 20% второй аттестации. Quiz II - отдельная работа. **Сроки:** по объявлению в Moodle.

1. Цель и связь с силлабусом

Реализуйте шесть частей с восемью командами: LCS, упорядоченное слияние, выбор вершин дерева, три поиска одного шаблона, суффиксный индекс и поиск словаря. Это самостоятельные модули сравнения последовательностей, планирования и обработки текста; состояние между ними не переносится.

Недели 8-10: subsequence/partition/tree DP; KMP и полиномиальный rolling hash; suffix array, Kasai LCP и Aho-Corasick. Fibonacci и 0/1 Knapsack используются для подготовки, дополнительных обязательных реализаций не требуется. Слияние применяет интервальный шаблон лекции 8; формула стоимости отличается от matrix-chain multiplication.

Java 17+ - основной язык силлабуса. Выберите один поддерживаемый язык курса: Java 17+, Python 3.12+, C++ 17 или C#/.NET 8. Стандартные коллекции, сортировка и предоставленный JSON-адаптер разрешены. Алгоритмы реализуйте самостоятельно; готовые алгоритмические библиотеки и встроенный поиск подстроки не заменяют требуемые методы.

2. Общий контракт программы

UTF-8 JSON Lines: один объект запроса на строку, один объект результата на строку. В stdout только результаты. Один процесс может получить последовательность разных запросов; состояние каждого запроса сбрасывается. Строки содержат печатные ASCII-символы 32-126, регистр значим, Unicode-нормализация не применяется. Пробелы и пунктуация - обычные символы.

Позиции символов, исходных блоков и ID вершин начинаются с 0. Обязательные числовые поля входа/выхода - целые числа, не JSON Boolean и не дробные числа. Стоимости, суммарные веса и счётчики представляйте точно с безопасной 64-битной целочисленной арифметикой; Python integers подходят. Порядок входных массивов сохраняйте там, где он определяет выходные позиции. Порядок полей JSON произвольный. Входы удовлетворяют условиям, некорректный JSON не оценивается. Если оптимальный ответ неоднозначен, принимается любой допустимый оптимум.

Описание полей ниже использует обозначения типов; примеры содержат настоящий корректный JSON. В каждом ответе обязательны названные поля с правильными типами, даже если проверяется только значение оптимума. Дополнительные поля объекта текущие валидаторы игнорируют; добавлять их не требуется.

Оценки сложности относятся к алгоритму после разбора JSON. Вспомогательную память отделяйте от памяти входа/выхода. Для поиска строк n - длина текста, m - длина шаблона, z - число возвращаемых позиций; сам вывод z позиций требует как минимум z операций вывода. Пустые входы обрабатываются явно.

Всего шесть частей, восемь команд и девятнадцать групп оценивания. В части 4 три реализации: naive, kmp и rk; suffix объединяет построение суффиксного массива и Kasai LCP.

Часть 1: Сравнение последовательностей - LCS

Сопоставьте две версии последовательности и найдите общие символы с сохранением порядка.

Команда: `lcs`

Вход: `{task:lcs,a:string,b:string}`

Выход: `{length,sequence:string}`

Ограничения: 0 ≤ |a|, |b| ≤ 1000; |a|·|b| ≤ 1000000.

Реализуйте динамическое программирование по префиксам и восстановите одну наибольшую общую подпоследовательность. sequence должна быть подпоследовательностью обеих строк и иметь оптимальную длину. Символы не обязаны идти подряд. При нескольких оптимумах принимается любой. Если хотя бы одна строка пуста, length=0 и sequence="". Объясните состояние, переходы, базовые случаи и восстановление.

Сложность: O((|a|+1)(|b|+1)) времени и памяти разрешены; такая запись учитывает пустые строки.

Пример входа:

```
{
  "task": "lcs",
  "a": "tea_time",
  "b": "team"
}
```

Один допустимый результат:

```
{
  "length": 4,
  "sequence": "team"
}
```

В tea_time символ m находится после i: team - подпоследовательность, но не подстрока.

Объясните: Почему при несовпадении последних символов достаточно сравнить два меньших префикса? Почему восстановленный ответ имеет оптимальную длину?

Часть 2: Слияние упорядоченных блоков - интервальное DP

Найдите самый дешёвый порядок слияния соседних блоков, сохраняя исходный порядок.

Команда: `merge`

Вход: `{task:merge,sizes:[...]}`

Выход: `{cost,plan:[[l,k,r],...]}`

Ограничения: 0 ≤ n ≤ 200; 0 ≤ sizes[i] ≤ 1000000000; n - длина sizes.

Каждый текущий блок представляет замкнутый интервал исходных индексов. Разрешено объединить существующие соседние блоки [l, k] и [k+1, r], заплатив сумму их размеров, и заменить их блоком [l, r]. Верните минимальную общую стоимость и хронологический план [l, k, r]. Изначально блоки равны [i, i]. При n≤1: cost=0, plan=[]. При n>1 план содержит n-1 слияний. Принимается любой оптимальный план. Выбор двух произвольных минимальных блоков нарушает требование смежности.

Сложность: O(n^3) времени, O(n^2) памяти; для пустого входа O(1). Продвинутые оптимизации DP не требуются.

Пример входа:

```
{
  "task": "merge",
  "sizes": [2, 3, 4]
}
```

Один допустимый результат:

```
{
  "cost": 14,
  "plan": [[0, 0, 1], [0, 1, 2]]
}
```

Сначала объединяются размеры 2 и 3: стоимость 5; затем 5 и 4: стоимость 9. Итого 14, а другой порядок даёт 16.

Объясните: Что означает dp[l][r]? Почему последний split делит оптимальный план на два оптимальных подплана? Почему выводить план нужно после планов детей?

Часть 3: Выбор совместимых вершин - Tree DP

Выберите набор максимального веса, если соседние вершины несовместимы, а связи образуют дерево.

Команда: `tree`

Вход: `{task:tree,n,weights:[...],edges:[[u,v],...]}`

Выход: `{weight,selected:[vertex IDs,...]}`

Ограничения: n - целое от 0 до 100000; weights содержит ровно n целых чисел, -1000000000 ≤ weights[i] ≤ 1000000000. При n>0 edges задаёт связное неориентированное дерево из n-1 различных рёбер; концы - целые от 0 до n-1. При n=0 оба массива пусты. Граница одного weights[i] не ограничивает суммарный вес ответа.

Реализуйте максимальное по суммарному весу независимое множество методом take/skip DP на дереве и восстановите выбранные вершины. Соседние вершины нельзя выбирать одновременно; каждая выбранная вершина встречается один раз. Пустое множество разрешено, поэтому при всех отрицательных весах ответ 0. При n=0 верните weight=0 и selected=[]. Порядок selected произвольный; при нескольких оптимумах принимается любой. Выбор корня не меняет оптимальный вес. Обработайте длинную цепочку без переполнения стека.

Сложность: O(n) времени и памяти для n>0, O(1) для пустого дерева.

Пример входа:

```
{
  "task": "tree",
  "n": 3,
  "weights": [4, 9, 6],
  "edges": [[0, 1], [1, 2]]
}
```

Один допустимый результат:

```
{
  "weight": 10,
  "selected": [0, 2]
}
```

Вершины 0 и 2 не смежны и дают 4+6=10; выбор только центральной даёт 9.

Объясните: Почему take[u] использует только skip детей, а skip[u] - максимум двух состояний? Как решение родителя ограничивает восстановление?

Часть 4: Точный поиск - наивный алгоритм

Получите простой точный поиск для сравнения результатов и времени других методов.

Команда: `naive`

Вход: `{task:naive,text:string,pattern:string}`

Выход: `{positions:[...]}`

Ограничения: 0 ≤ |text|, |pattern| ≤ 3000.

Реализуйте посимвольное сравнение шаблона с каждым допустимым окном. Верните ВСЕ начальные индексы в строго возрастающем порядке, включая перекрытия. Пустой шаблон совпадает на каждой границе 0..|text| включительно; шаблон длиннее текста даёт []. Не заменяйте алгоритм встроенным поиском.

Сложность: При n=|text|, m=|pattern|: O(n·m+n+1) времени после разбора входа, включая вывод; O(z+1) памяти результата для z позиций и O(1) вспомогательной памяти. Пустой шаблон даёт n+1 позиций.

Пример входа:

```
{
  "task": "naive",
  "text": "aaaaa",
  "pattern": "aaa"
}
```

Один допустимый результат:

```
{
  "positions": [0, 1, 2]
}
```

Вхождения aaa в aaaaa начинаются с 0, 1 и 2; перекрытия разрешены.

Объясните: Как проверить все допустимые старты и не потерять последний? Чем поиск всех вхождений отличается от первого?

Часть 4: Точный поиск - KMP

Используйте уже известные совпадения префиксов при продолжении поиска.

Команда: `kmp`

Вход: `{task:kmp,text:string,pattern:string}`

Выход: `{positions:[...]}`

Ограничения: 0 ≤ |text| ≤ 200000; 0 ≤ |pattern| ≤ 100000.

Реализуйте префикс-функцию и KMP. Контракт совпадений такой же, как у naive, включая пустой шаблон, порядок индексов и перекрытия. После полного совпадения продолжайте с префиксным fallback. Встроенный поиск не заменяет реализацию KMP. На общем диапазоне входов сравните результаты naive/KMP/RK.

Сложность: O(n+m+z+1) времени, O(m+1) вспомогательной памяти и O(z+1) памяти результата; n=|text|, m=|pattern|, z - число позиций. Пустой шаблон обрабатывается до обращения к его символам и даёт n+1 границ.

Пример входа:

```
{
  "task": "kmp",
  "text": "abababa",
  "pattern": "aba"
}
```

Один допустимый результат:

```
{
  "positions": [0, 2, 4]
}
```

После найденного aba нельзя безусловно обнулять совпавший префикс: можно потерять старты 2 и 4.

Объясните: Что означает pi[i]? Почему fallback pi[j-1] сохраняет возможные совпадения и не требует возвращать индекс текста назад?

Часть 4: Точный поиск - Rabin-Karp

Отбрасывайте окна с другим хешем и проверяйте символы каждого кандидата с равным хешем.

Команда: `rk`

Вход: `{task:rk,text:string,pattern:string}`

Выход: `{positions:[...]}`

Ограничения: 0 ≤ |text|, |pattern| ≤ 10000; |text|·|pattern| ≤ 20000000.

Реализуйте полиномиальный rolling hash: base=31, modulus=101, значения символов - ASCII-коды, наибольшая степень у левого символа. Это учебная конфигурация для наблюдения коллизий. Нормализуйте отрицательные остатки. При равных хешах сравнивайте реальные символы перед добавлением позиции. Контракт совпадений общий с naive/KMP. Встроенный поиск не заменяет алгоритм. Достаточно 64-битной арифметики.

Сложность: При 1≤m≤n: O(n+m+c·m), где c - число окон-кандидатов с равным хешем; возможен квадратичный случай. Общая оценка для всех допустимых входов: O(n·m+n+m+1), включая n+1 позиций пустого шаблона. Обработайте m=0 и m>n до построения окна. Для rolling windows разрешено O(m+1) вспомогательной памяти плюс результат; для prefix hashes - O(n+m+1) плюс результат. Безусловная линейная оценка неверна.

Пример входа:

```
{
  "task": "rk",
  "text": "abdj",
  "pattern": "ab"
}
```

Один допустимый результат:

```
{
  "positions": [0]
}
```

ab и dj имеют хеш 75; только окно ab является совпадением. Это проверка коллизии, а не допустимый ложноположительный ответ.

Объясните: Как обновляется хеш окна? Почему совпадение хешей не доказывает равенство строк? Что меняется во времени при множестве кандидатов?

Часть 5: Индекс повторов - суффиксный массив и Kasai LCP

Упорядочьте суффиксы и используйте общие префиксы соседей для поиска повторяющихся подстрок.

Команда: `suffix`

Вход: `{task:suffix,text:string}`

Выход: `{sa:[...],lcp:[...],repeated:string}`

Ограничения: 0 ≤ |text| ≤ 50000. В выходе только непустые суффиксы исходного текста; добавленный sentinel и пустой суффикс не выводятся.

Постройте суффиксный массив методом prefix doubling, затем LCP алгоритмом Касаи. sa - перестановка 0..n-1 в лексикографическом порядке суффиксов. lcp имеет длину n: lcp[0]=0, lcp[i] сравнивает суффиксы sa[i-1] и sa[i]. repeated - любая самая длинная подстрока с двумя различными началами; перекрытия разрешены. Если повтора нет, repeated="". При пустом тексте sa=[], lcp=[], repeated="". Не создавайте в памяти все строки суффиксов.

Сложность: При n≥1 prefix doubling с comparison sort занимает O(n log²(n+1)); линейная counting/radix-сортировка rank pairs за раунд позволяет O(n log(n+1)). Kasai - O(n). Вспомогательная память и память результата - O(n+1). Пустой вход - O(1).

Пример входа:

```
{
  "task": "suffix",
  "text": "banana"
}
```

Один допустимый результат:

```
{
  "sa": [5, 3, 1, 0, 4, 2],
  "lcp": [0, 1, 3, 0, 0, 2],
  "repeated": "ana"
}
```

ana встречается с началами 1 и 3; длина 3 равна max(lcp). Эти вхождения перекрываются.

Объясните: Что кодирует rank pair? Чем порядок суффиксов отличается от порядка их исходных индексов? Почему max LCP даёт длину самого длинного повтора?

Часть 6: Поиск словаря шаблонов - Aho-Corasick

Постройте один автомат словаря и посчитайте вхождения шаблонов в независимых текстах.

Команда: aho

Вход: `{task:aho,patterns:[nonempty strings,...],texts:[strings,...]}`

Выход: `{counts:[[count for each dictionary position],...]}`

Ограничения: 0 ≤ число patterns ≤ 10000; общая длина patterns ≤ 100000; каждый шаблон непустой. 0 ≤ число texts ≤ 100; общая длина texts ≤ 200000; отдельный текст может быть пустым.

Один trie/автомат Aho-Corasick с failure links на запрос. Для каждого текста начинайте состояние и счётчики заново. Учитывайте перекрытия и шаблоны-суффиксы. Дубликаты словаря - отдельные позиции с одинаковым счётчиком. counts содержит строку на каждый текст и столбец на каждый элемент patterns в исходном порядке. Пустой словарь даёт [] для каждого текста; пустой texts даёт []. Пустой текст даёт нули. Не заменяйте автомат отдельными KMP-поисками. Допустимы распространение посещений по failure links или

корректный обход output links; не копируйте квадратично растущие списки унаследованных выходов. Требуется матрица счётчиков, а не список отдельных совпадений. Одинаковые шаблоны могут разделять терминальное состояние; не создавайте запись на каждое вхождение каждой копии словаря. На плотных совпадениях используйте распространение посещений по failure links в обратном порядке или столь же эффективное агрегирование: буквальный обход всех совпадений может не уложиться во время, даже считая их правильно.

Сложность: L - суммарная длина шаблонов с дубликатами, p - размер словаря, S - число состояний trie с корнем. Для фиксированного ASCII из 95 символов построение занимает O(L+95·S+p) времени и O(95·S+p) вспомогательной памяти. На каждый независимый текст распространение посещений требует O(|text|+S+p). Обход output links вместо этого требует O(|text|+z+p), где z - число посещённых терминальных совпадений, и не гарантирует прохождение плотных случаев по времени. Матрица ответа занимает O(q·(p+1)+1) для q текстов; при p=0 сохраняется пустая строка на каждый текст. Не копируйте квадратично растущие списки унаследованных выходов.

Пример входа:

```
{
  "task": "aho",
  "patterns": ["he", "she", "he"],
  "texts": ["sheshe", "sheshe", ""]
}
```

Один допустимый результат:

```
{
  "counts": [[2, 2, 2], [2, 2, 2], [0, 0, 0]]
}
```

he - суффикс she; оба шаблона учитываются. Второй одинаковый текст не накапливает счётчики первого.

Объясните: Что сохраняет failure link? Как учесть шаблон-суффикс и дубликаты? Почему распространение счётчиков идёт в обратном BFS-порядке?

Оценивание и защита

Практика оценивается из 50 независимыми автоматическими проверками опубликованных групп требований. Защита - отдельный индивидуальный квиз из 50. Требуемые методы, анализ, собственные тесты и benchmark остаются учебными результатами без скрытых ручных баллов и произвольных вычетов. По результату программы нельзя доказать внутренний алгоритм или авторство; понимание проверяет защита. Веса и исправление ошибок описаны в RUBRIC.md.

Защита на английском: 15 вопросов за 25 минут, одна попытка; 10 вопросов с одним ответом из четырёх вариантов и 5 с несколькими ответами из пяти вариантов. Максимум вопроса - 1 сырой балл. Оценка квиза = raw/15×50. Верный одиночный ответ даёт 1, неверный или пустой - 0. Вопросы требуют применить методы к новым коротким примерам: объяснить переход, проверить восстановленный ответ, найти контрпример, диагностировать ошибку реализации или обосновать сложность.

При k правильных вариантах checkbox (k=2 или 3) верная отметка даёт 1/k, неверная вычитает 1/(2k). Отмеченные веса суммируются; минимум за вопрос 0. Для двух правильных: +0,50/-0,25; для трёх: +1/3/-1/6. Выбор всех пяти поэтому даёт 0,25 или около 0,67 соответственно; полный балл требует всех правильных без неправильных. Это правило задания по последнему формату A3, а не требование силлабуса. Индивидуальное устное восстановление баллов не предусмотрено. Ошибочный вопрос, ключ или тест исправляется единообразно для всех затронутых работ по правилам курса.

Во время квиза работайте самостоятельно, без ИИ и внешней помощи. Соблюдайте инструкции и утверждённые условия доступности. Три страницы по пять вопросов, можно возвращаться к предыдущим. Даты, доступ и выпуск ответов объявляются в Moodle. Ключи квиза не входят в публичный комплект задания.

Анализ, тесты и сравнение

Заполните ANALYSIS.md на английском. Для каждой команды: состояние/инвариант, переходы или обновления, базы, порядок вычисления, корректность, восстановление при необходимости, время и память. Для собственных тестов запишите вход, ожидаемый и фактический результат, обнаруживаемую ошибку. По применимости проверьте пустые входы, равные оптимумы, перекрытия, большие значения и последовательные запросы. Указывайте только реальные запуски и измерения.

Сравните naive/KMP/RK на одинаковых парах text/pattern трёх возрастающих размеров в общем диапазоне: обе длины≤3000 удовлетворяют всем трём контрактам. Укажите воспроизводимый генератор/seed, n, m, распределение шаблона, версию runtime, команды, границу замера, минимум три измерения каждого метода и медианы. Перед замером проверьте совпадение правильных ответов. Компиляция исключается, граница замера одинакова. Опишите прогрев/JIT и оборудование. Отдельно рассмотрите random и repetitive cases. Учебный modulus 101 не представляет все production-реализации хеширования. Известная коллизия - тест корректности; медианы не доказывают асимптотику.

Запуск, проверка и сдача

Создайте свой репозиторий из официального A4 template в Moodle. Выберите язык в submission.json. Реализуйте методы соответствующей source-папки, сохраняя предоставленный транспорт. Команды существующего A4 launcher:

```
python tools/run.py --language Java --setup
python tools/run.py --language Java --check
python tools/run.py --language Java --input examples/lcs.jsonl
```

Вместо Java используйте Python, Cpp или CSharp. Для launcher нужен Python 3.12+ при любом языке решения. В Windows можно использовать py -3. Без --language берётся submission.json. setup проверяет запуск; TODO намеренно не реализуют алгоритмы. Public checks содержат 44 случая и повторяют полный пакет дважды в одном процессе для проверки сброса состояния. Успешный setup сам по себе не проверяет результаты алгоритмов.

В коммит входят исходники выбранного языка, submission.json, собственные тесты tests/student, английский ANALYSIS.md и сведения о сдаче. Выполните локальные/public checks; перед сдачей реализация должна проходить public checks. В Moodle Online text вставьте ссылку точного коммита, по желанию ссылку Actions run того же коммита, затем нажмите Submit assignment и проверьте статус. Ссылка коммита содержит полный 40-символьный SHA и определяет оцениваемую версию. Для private repository предоставьте доступ преподавателю. Сроки и согласованные продления берутся из Moodle. Public checks дают обратную связь и не выставляют/не переносят официальную оценку практики.

Официальные изолированные проверки: 60 секунд на компиляцию, 20 секунд на пакет группы требований, 1 CPU, 768 MiB памяти контейнера (Java heap≤512 MiB),128 процессов, максимум 8 MiB UTF-8 вывода. Каждая группа запускается одним JSONL-пакетом. Ошибка процесса, timeout или некорректная/отсутствующая строка результата может провалить весь пакет; корректно оформленный, но неверный ответ влияет на свой случай. Решение поддерживает полные области входов выше; прохождения открытых примеров недостаточно для такого вывода. Медианы времени не используются для рейтинга.

Академическая честность и материалы

Следуйте силлабусу. В ANALYSIS.md укажите внешние источники и инструменты ИИ, их вклад, использованный материал и собственную проверку; если не использовались - напишите это. Вы отвечаете за корректность, оригинальность и качество. По одному стилю письма нельзя установить нарушение.

Основа: недели 8-10 силлабуса и файлы лекций 8 (DP),9 (KMP/hash),10 (suffix/Kasai/Aho). В LECTURE_CLARIFICATIONS_EN_RU.md объяснены индексы, коллизии, sentinel и сложность. Обязательны только перечисленные выше методы; продвинутые DP-оптимизации и линейное построение suffix array не нужны. Решение должно поддерживать указанные выше области входов.

Приложение: примеры и соглашения лекций

- dp[i][j] в LCS относится к первым i и j символам; при i, j>0 сравниваются a[i-1] и b[j-1]. Нулевая строка и столбец - база. Краткая формула лекции смешивает соглашения об индексах.
- merge использует partition pattern лекции 8. Добавочная стоимость split - сумма sizes[l..r]; matrix-chain использует размеры матриц. Формулы стоимости различаются.
- Tree MWIS соответствует take/skip лекции 8. Выбор корня нужен для вычисления и не меняет оптимум.
- У файла Lecture 9 ошибочный титул Lecture 10 и посторонний слайд Kosaraju. Он не добавляет графовых требований к A4.
- В псевдокоде naive внутренний цикл сравнивает j=0..m-1, а не 0..m включительно. При m≤n допустимы старты i=0..n-m. Пустой шаблон обрабатывается отдельно по правилу n+1 границ A4.
- modulus 101 выбран для проверки коллизий. После равенства хешей проверяются символы. Точный RK не гарантирует линейное время на всех входах.
- В лекции 10 banana$ включает sentinel. В A4 выводятся только непустые суффиксы исходного текста; внутренний sentinel в ответ не входит.
- В примере Aho для ushers в лекции 10 ошибочно указаны позиции. При нумерации с нуля и включённых концах: she - 1..3, he - 2..3, hers - 2..5. he и she заканчиваются на одном символе, hers - позже. Начало вхождения не равно его концу.
- Doubling с comparison sort даёт O(n log²n); линейная сортировка rank pairs на раунд позволяет O(n log n). Kasai - O(n).
- DP не всегда даёт полиномиальную сложность по битовой длине входа: O(nW) для knapsack псевдополиномиально. Сравнение memoization/tabulation зависит от посещаемых состояний, памяти и реализации.